

f-String / Formatted String Literal

A formatted string literal or f-string is a string literal that is prefixed with *f* or *F*.

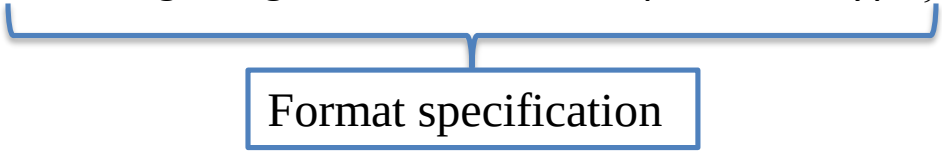
These strings may contain replacement fields, which are expressions delimited by curly braces {}. While other string literals always have a constant value, formatted strings are really expressions evaluated at run time.

Syntax:- `f“{index/key/name:[fill][align][sign][#][0][width][,][.precision]type}”`

Ex:-

`a = 10`

`print(f“{a}”)`



Format specification

The diagram illustrates the format specification part of the f-string syntax. A blue bracket is drawn under the portion of the syntax string `[fill][align][sign][#][0][width][,][.precision]type` that includes `[fill][align][sign][#][0][width][,][.precision]`. A blue box is then drawn around the text "Format specification", which is positioned below the bracketed portion of the syntax string.

Ex:- `print(f“My age is {a}”)`

f-String / Formatted String Literal

Conversion Type	Meaning
d	Signed integer decimal.
o	Signed octal value.
x	Signed hexadecimal (lowercase).
X	Signed hexadecimal (uppercase).
b	Binary Format
e	Floating point exponential format (lowercase).
E	Floating point exponential format (uppercase).
f	Floating point decimal format. (Default: 6)
F	Same as 'f'. Except displays 'inf' as 'INF' and 'nan' as 'NAN'

f-String / Formatted String Literal

Conversion Type	Meaning
c	Character. Converts the integer to the corresponding Unicode character before printing
g	General format. Rounds number to p significant digits. (Default precision: 6)
G	Same as 'g'. Except switches to 'E' if the number is large.
n	Same as 'd'. Except it uses current locale setting for number separator
s	String (converts any Python object using str()).
%	Percentage. Multiplies the number by 100 and displays in fixed ('f') format, followed by a percent sign.

f-String / Formatted String Literal

Alignment Type	Meaning
<	Forces the field to be left-aligned within the available space (This is default for most objects)
^	Forces the field to be centered within the available space.
>	Forces the field to be right-aligned within the available space (This is default for Numbers)
=	Forces the padding to be placed after the sign (if any) but before the digits. This is used for printing fields in the form '+000000120'. This alignment option is only valid for numeric types. It becomes the default when '0' immediately precedes the field width.

f-String / Formatted String Literal

Sign	Meaning
+	indicates that a sign should be used for both positive as well as negative numbers.
-	indicates that a sign should be used only for negative numbers (this is the default behavior).
	(a space) indicates that a leading space should be used on positive numbers, and a minus sign on negative numbers.

f-String / Formatted String Literal

- The '#' option causes the “alternate form” to be used for the conversion. The alternate form is defined differently for different types. This option is only valid for integer, float, complex and Decimal types. For integers, when binary, octal, or hexadecimal output is used, this option adds the prefix respective '0b', '0o', or '0x' to the output value.
- The ',' option signals the use of a comma for a thousands separator. For a locale aware separator, use the 'n' integer presentation type instead.
- The '_' option signals the use of an underscore for a thousands separator for floating point presentation types and for integer presentation type 'd'.

f-String / Formatted String Literal

`:[fill][align][sign][#][0][width][,][.precision]type`

`print(f"{num:5d}")`

			1	5
--	--	--	---	---

`print(f"{num:05d}")`

0	0	0	1	5
---	---	---	---	---

`print(f"{num:+5d}")`

		+	1	5
--	--	---	---	---

`print(f"{num:<5d}")`

1	5			
---	---	--	--	--

`print(f"{num:*<5d}")`

1	5	*	*	*
---	---	---	---	---

`print(f"{num:*>5d}")`

*	*	*	1	5
---	---	---	---	---

`print(f"{num:^5d}")`

	1	5		
--	---	---	--	--

f-String / Formatted String Literal

:`[fill][align][sign][#][0][width][,][.precision]`type

```
print(f"{num:8f}")
```

1	5	.	6	5	0	0	0	0
---	---	---	---	---	---	---	---	---

```
print(f"{num:8.3f}")
```

		1	5	.	6	5	0
--	--	---	---	---	---	---	---

```
print(f"{num:+8.2f}")
```

		+	1	5	.	6	5
--	--	---	---	---	---	---	---

```
print(f"{num:<8.2f}")
```

1	5	.	6	5			
---	---	---	---	---	--	--	--

```
print(f"{num:*<8.2f}")
```

1	5	.	6	5	*	*	*
---	---	---	---	---	---	---	---

```
print(f"{num:*>8.2f}")
```

*	*	*	1	5	.	6	5
---	---	---	---	---	---	---	---

```
print(f"{num:^8.2f}")
```

	1	5	.	6	5		
--	---	---	---	---	---	--	--

f-String / Formatted String Literal

:
[fill][align][sign][#][0][width][,][.precision]type

name = "Geek"

```
print(f"{name:8s}")
```

G	e	e	k				
---	---	---	---	--	--	--	--

```
print(f"{name:<8}")
```

G	e	e	k				
---	---	---	---	--	--	--	--

```
print(f"{name:*<8}")
```

G	e	e	k	*	*	*	*
---	---	---	---	---	---	---	---

```
print(f"{name:>8}")
```

				G	e	e	k
--	--	--	--	---	---	---	---

```
print(f"{name:*>8s}")
```

*	*	*	*	G	e	e	k
---	---	---	---	---	---	---	---

```
print(f"{name:^8s}")
```

		G	e	e	k		
--	--	---	---	---	---	--	--

f-String / Formatted String Literal

`:[fill][align][sign][#][0][width][,][.precision]type`

`name = "GeekyShows"`

`print(f"{name:.3s}")`

G	e	e
---	---	---

`print(f"{name:8.3s}")`

G	e	e					
---	---	---	--	--	--	--	--

`print(f"{name:*<8.3s}")`

G	e	e	*	*	*	*	*
---	---	---	---	---	---	---	---

`print(f"{name:>8.3s}")`

					G	e	e
--	--	--	--	--	---	---	---

`print(f"{name:*>8.3s}")`

*	*	*	*	*	G	e	e
---	---	---	---	---	---	---	---

`print(f"{name:^8.3s}")`

		G	e	e			
--	--	---	---	---	--	--	--